Coding Assignments #5 (Part Two)

August 4, 2025

Directions:

1. You will be penalized for not following each of the directions exactly as written below. If you aren't sure about what something means, ask the professor!
2. Put all your work in a single Haskell file.
3. Submit your work (before the deadline) to Canvas.
4. **Important: You must avoid global helper functions**. Keep the scope of your helper functions limited by using `let`s and `where`s.
5. **Important:** You will be graded on excessive use of helper functions when patterns or guards would make more sense.
6. **Important:** You will lose points **each time** you use the `if` keyword. Use patterns and guards instead!

1. (Call your function **splitWordsr**. Using only techniques from chapters 1-5, write a function that takes in a String and splits it into a list of words. For example,

```
splitWordsr "this is a string that fits on one line"
```

   will return (split to two lines for formatting reasons)

```
["this", "is", "a", "string", "that", "fits", "on",
        "one", "line"]
```

   You must use `foldr` or `foldr1` at some point in your calculation.

   Hints: First, Write a (**it's OK if its global in this case**) helper function that takes (1) a list of Strings and (2) a character and returns a list of Strings. If the first character of the first String in the input list is a space, add the input character as a new string at the beginning of the list.

Otherwise, prepend the input character to the first character in the input list.

Second, once you have the helper function written, you should only need functions like `map`, `reverse`, `flip`, `foldr`, and `filter`; along with basic arithmetic operations.

2. (Call your function **splitWordsl**). Using only techniques from chapters 1-5, repeat the previous question but use a `foldl` instead of `foldr`.

3. (Call your function **blockTextr**). Using only techniques from chapters 1-5, write a function that takes in three parameters: `n`: an `Int` which specifies the maximum amount of characters allowed on a single line; `sentence`: a `String` which needs to be split up into a list of strings. The sum of the number of characters in each list should be strictly less than `n`. For example,

```
blockTextr 20 "this is a string that fits on a line"
```

   Should result in (split to two lines for formatting reasons):

```
[["this", "is", "a", "string", "that"],["fits",
          "on", "a", "line"]]
```

   Hints: First, write a helper function (**It's OK if its global in this case**) that processes a single String: either it adds it to a list of `String`s (the most recent line being processed) or it starts a new line (a new list of `String`s) if the `String` is too long. The inputs and output of the function are an `Int` (one more than the number of characters allowed on a line); a list of list of `String`s (the strings already processed by `blockTextr`); a `String` (the next string to include in the list of list of `String`s; and produces a list of list of `String`s which is the same as the input list of lists, but includes the single `String` argument as well.

   Second, you should only need the helper function, `foldr`, `reverse` and `map`; along with arithmetic operators.

4. (Call your function **blockTextl**). Using only techniques from chapters 1-5, write a function that does the same thing as the function from the previous question, but make sure to use `foldl` instead of `foldr`.